

CYBERSECURITY

LAB 4

I. Introduction.

The laboratory presents issues related to asymmetric encryption. Asymmetric key cryptography relies on one key for encryption and a different but related key for decryption. It is very important to note that even with knowledge of the cryptographic algorithm and the encryption key, it is computationally infeasible to determine the decryption key. The advantage of the system is that each communicating party needs just a key pair for communicating with any number of other communicating parties. Besides, some algorithms, such as RSA, also exhibit an important characteristic, i.e., either of the two related keys can be used for encryption, with the other used for decryption.

Asymmetric cryptography is based on one-way functions - those that can be easily calculated in one direction, but very difficult to calculate in the other. There are two types of asymmetric algorithms. The first one, which is called "factoring problem," whose encryption security is based on the difficulty of factoring the product of two large prime numbers. For example, multiplication is easy and distribution into factors (factorization) is difficult (on which RSA is based, for example). The second type is algorithms based on elliptical curve cryptography whose security is based on the computational complexity of discrete logarithms on elliptical curves - known as Elliptic Curve Discrete Logarithm Problem (ECDLP). Modulo amplification is easy, and discrete logarithming is difficult (this applies to such algorithms as ElGamal, DSA, and ECC).

The main interest of the laboratory is the RSA algorithm. The RSA algorithm is the most popular asymmetric key cryptographic algorithm. The scheme is a block cipher. In this case, a plain text and a ciphertext are integers from 0 to N . A typical size for N is 1024 bits or 309 decimal digits, however, currently, 4096 bits are considered to be safe. This means that N is less than 2^{1024} (for N equal 1024 bits). The principle of the algorithm is explained below.

Contrary to popular belief, Diffie-Hellman was the first publicly described asymmetric algorithm (not RSA). This cryptographic protocol allows two entities to establish a common key on an insecure channel. In other words, Diffie-Hellman is often used to allow actors to exchange a symmetrical key via some insecure medium such as the Internet. Despite the passage of time, the DH protocol is still widely used, for example in the most recent version of TLSv 1.3 (Transport Layer Security). ElGamal is based on the Diffie-Hellman key exchange algorithm and is used in some versions of Pretty Good Privacy (PGP).

The process of asymmetric cryptography is visualized in Figure 1. With public-key/asymmetric cryptography, Alice will get Bob's public key and use that to encrypt the message she sends to Bob. If Eve intercepts the message and gains access to Bob's public key, that's OK, because that key won't decrypt the message. Only Bob's private key will do so, and this he safeguards.

LAB 4

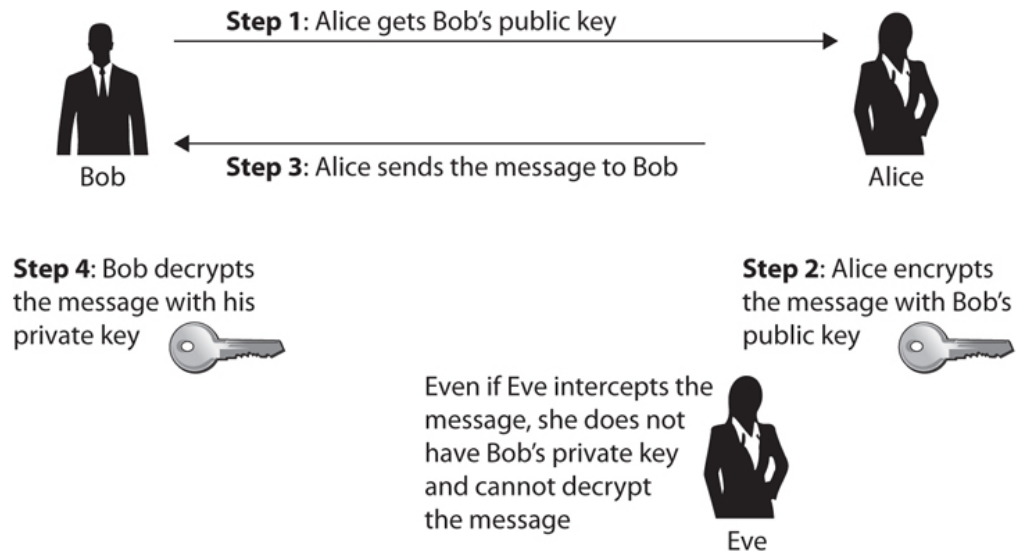


Figure 1 Exchange of messages with asymmetrical cryptography.

Description of asymmetrical cryptography from a mathematical point of view. A plain text is encrypted in blocks. For some blocks of plain text M and encryption block C , the whole process can be summarised as follows:

1. Select two large prime numbers P and Q , such that $P \neq Q$.
2. Calculate $N = P \times Q$.
3. Calculate $\phi(N) = (P-1)(Q-1)$.
4. Select an integer E as the public key (i.e., the encryption key) such that it is not a factor of $\phi(N)$.
5. Calculate D as the private key (i.e., the decryption key) such that:

$$(D \times E) \bmod \phi(N) = 1$$

6. From the plain text, M the ciphertext is calculated as:

$$C = M^E \bmod N$$

7. This ciphertext is sent to the receiver for decryption.
8. The ciphertext is decrypted to the plain text as:

$$M = C^D \bmod N$$

II. Tasks

1. Properties of the RSA algorithm.

- 1.1. See the demonstration of the RSA algorithm (Encryption/Asymmetric/Demonstration RSA).
- 1.2. Generate the following cryptographic keys (Digital Signature/PKI/Generate/Import keys):
 - RSA length 512 bits

LAB 4

- RSA length 1024 bits
 - RSA length 2048 bits
- 1.3. For different key lengths (512, 1024, 2048) and previously prepared three plaintexts (homogeneous, medium-diversified and highly diversified text - at least 1500 characters), compare the entropy of plaintext and ciphertext. Check and compare the autocorrelation of the ciphertext and plaintext.
 - 1.4. For different RSA algorithm key lengths, measure the encryption and decryption time for files of 1MB, 2MB, 5MB (with an accuracy of 1 seconds).
 - 1.5. Encrypt and decrypt the plaintexts from point 4 using a symmetric algorithm, try to notice the time of these operations.
 - 1.6. Introduce the following changes to the ciphertext (use a different length of a key). Then, decrypt the ciphertexts and check data integrity:
 - change the value of 1 byte,
 - remove 1 byte,
 - remove several (several dozen) bytes,
 - remove a piece of ciphertext equal to the length of the algorithm module (512, 1024, ...).

Questions:

- 1.7. Does the key length affect the ciphertext entropy? If so, how?
- 1.8. Does the length of the key affect the autocorrelation of the ciphertext? If so, how?
- 1.9. Does the ciphertext entropy depend on the plaintext entropy? If so, how?
- 1.10. How does the encryption/decryption time depend on the file length?
- 1.11. What is the time of encryption/decryption with asymmetric algorithms compared to the execution of these operations with a symmetric algorithm?
- 1.12. What do the changes in the plaintext look like when you introduce the ciphertext falsification? What is their scale? What does it depend on?
- 1.13. Is it possible to remove a fragment from the ciphertext so that the remaining text of the plaintext after decryption is readable?
- 1.14. What are the advantages and disadvantages of asymmetric algorithms compared to symmetric ones?
- 1.15. In which applications is it better to use asymmetric algorithms and in which applications is it better to use symmetric algorithms?